

《系统开发工具基础》实验报告



中国海洋大学
OCEAN UNIVERSITY OF CHINA

学号： 24120011027

姓名： 严程漪

专业： 物流管理

我的 github: [点击这里](#)

2026 年 9 月 16 日

目录

1 实验目的	2
1.1 学习 Packaging and Shipping Code	2
1.2 理解智能体编程	2
1.3 理解不至于代码	2
1.4 学习 Python 与 PyTorch	2
2 课内实验	2
2.1 第一题：从源码构建并在干净环境安装 Wheel	2
2.2 第二题：让编程智能体进入可验证的修复循环	3
2.3 第三题：把“无法处理”的协作材料改成可执行信息	4
2.4 第四题：修复一个可复现的线性回归训练循环	5
3 课外实例	6
3.1 实例一：用 printenv 对比虚拟环境激活前后的环境变量	6
3.2 实例二：写一个 Python 包，用 pyproject.toml 装进 venv，再生成锁文件	7
3.3 实例三：使用 Docker 与 Docker Compose 部署 Flask-Redis 多容器应用 .	9
3.4 实例四：注释查看	11
3.5 实例五：三个高 star 项目 README 对比分析	12
3.6 实例六：将 Python 包发布到 TestPyPI, 并构建 Docker 镜像	12
3.7 实例七：用 docker compose 构建班级网站	13
3.8 实例八：用 GitHub Pages 创建网站	14
3.9 实例九：为编程智能体创建 AGENTS.md、Skill 与子智能体	15
3.10 实例十：氛围编程	16
4 解题感悟	18

1 实验目的

1.1 学习 Packaging and Shipping Code

1.2 理解智能体编程

1.3 理解不至于代码

1.4 学习 Python 与 PyTorch

2 课内实验

2.1 第一题：从源码构建并在干净环境安装 Wheel

1、创建 src/greetlab/__init__.py、给定的 cli.py 和 pyproject.toml，并把“学号”替换为自己的学号。

```
ouc@islouc-vm:~$ cd q09
ouc@islouc-vm:~/q09$ mkdir -p src/greetlab
ouc@islouc-vm:~/q09$ touch src/greetlab/__init__.py
ouc@islouc-vm:~/q09$ cat << 'EOF' > src/greetlab/cli.py
> import argparse
> def main():
>     p = argparse.ArgumentParser()
>     p.add_argument("--name", required=True)
>     a = p.parse_args()
>     print(f"Hello, {a.name}!")
> EOF
ouc@islouc-vm:~/q09$ cat << 'EOF' > pyproject.toml
> [build-system]
> requires = ["setuptools>=68"]
> build-backend = "setuptools.build_meta"
> [project]
> name = "greetlab-24120011027"
> version = "0.1.0"
> [project.scripts]
> sdt-greet = "greetlab.cli:main"
> EOF
```

图 1: 文件创建

2、在已安装 build 工具的课程环境中运行 python -m build，生成 wheel。

```
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_24120011027-0.1.0.tar.gz and greetlab_24120011027-0.1.0-py3-none-any.whl
```

图 2: 生成 wheel 截图

3、新建一个干净虚拟环境，只从生成的 wheel 安装。

```
ouc@islouc-vm:~/q09$ python3 -m venv .venv
ouc@islouc-vm:~/q09$ source .venv/bin/activate
(.venv) ouc@islouc-vm:~/q09$ pip install dist/greetlab_24120011027-0.1.0-py3-none-any.whl
Processing ./dist/greetlab_24120011027-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-24120011027
Successfully installed greetlab-24120011027-0.1.0
```

图 3: 创建一个干净的虚拟环境

4、切换到 q09 之外运行 `sdt-greet --name<24120011027>`，确认输出正确。

```
(.venv) ouc@islouc-vm:~/q09$ cd ..  
(.venv) ouc@islouc-vm:~$ sdt-greet --name 24120011027  
Hello, 24120011027!
```

图 4: 结果截图

2.2 第二题：让编程智能体进入可验证的修复循环

1、复制 q09 为 q10，添加一个失败测试：当 name 只含空白字符时，main 应以 `SystemExit(2)` 结束。

```
ouc@islouc-vm:~$ cd q10  
ouc@islouc-vm:~/q10$ mkdir -p tests  
ouc@islouc-vm:~/q10$ cat << 'EOF' > tests/test_cli.py  
> import pytest  
> from greetlab.cli import main  
> def test_empty_name_raises_system_exit(monkeypatch):  
>     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])  
>     with pytest.raises(SystemExit) as exc_info:  
>         main()  
>     assert exc_info.value.code == 2  
> EOF
```

图 5: 失败测试

2、先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。

```
ouc@islouc-vm:~/q10$ python3 -m pytest  
===== test session starts =====  
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0  
rootdir: /home/ouc/q10  
collected 1 item  
  
tests/test_cli.py F [100%]  
  
===== FAILURES =====  
_____ test_empty_name_raises_system_exit _____  
  
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x7fa6574106a0>  
  
def test_empty_name_raises_system_exit(monkeypatch):  
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])  
    with pytest.raises(SystemExit) as exc_info:  
>         main()  
E         Failed: DID NOT RAISE <class 'SystemExit'>  
  
tests/test_cli.py:6: Failed  
----- Captured stdout call -----  
Hello,  !  
===== 1 failed in 0.02 seconds =====
```

图 6: 失败结果截图

```
ouc@islouc-vm:~/q10$ cat << 'EOF' > src/greetlab/cli.py
> import argparse
> def main():
>     p = argparse.ArgumentParser()
>     p.add_argument("--name", required=True)
>     a = p.parse_args()
>     if not a.name.strip():
>         p.error("Name cannot be empty")
>     print(f"Hello, {a.name}!")
> EOF
ouc@islouc-vm:~/q10$ python3 -m pytest
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q10
collected 1 item

tests/test_cli.py . [100%]

===== 1 passed in 0.01 seconds =====
```

图 7: 修改并运行

3、人工检查 diff，撤销无关修改，并再次运行测试。

```
ouc@islouc-vm:~/q10$ git diff --no-index ../q09/src/greetlab/cli.py src/greetlab/c
li.py
diff --git a/./q09/src/greetlab/cli.py b/src/greetlab/cli.py
index 99969f9..6ad6ae4 100644
--- a/./q09/src/greetlab/cli.py
+++ b/src/greetlab/cli.py
@@ -3,4 +3,6 @@ def main():
     p = argparse.ArgumentParser()
     p.add_argument("--name", required=True)
     a = p.parse_args()
+    if not a.name.strip():
+        p.error("Name cannot be empty")
+    print(f"Hello, {a.name}!")
ouc@islouc-vm:~/q10$ python3 -m pytest
===== test session starts =====
platform linux -- Python 3.8.10, pytest-4.6.9, py-1.8.1, pluggy-0.13.0
rootdir: /home/ouc/q10
collected 1 item

tests/test_cli.py . [100%]

===== 1 passed in 0.01 seconds =====
```

图 8: 人工检查 diff

由图可以看出，并没有无关修改

4、在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

```
ouc@islouc-vm:~/q10$ cat << 'EOF' > ai_log.md
> 核心提示：要求校验 --name 参数去除首尾空白，为空时抛出 exit code 2 错误
> 智能体改动：在 src/greetlab/cli.py 中加入 a.name.strip() 判断并调用 p.error()
> 人工验证：检查 diff 确认未修改其他文件，运行 pytest 确认测试成功通过
> EOF
```

图 9: ai_log.md

2.3 第三题：把“无法处理”的协作材料改成可执行信息

1、在 communication.md 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。

2、重写提交信息：标题使用祈使语气，正文说明问题和解决方案。

3、重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。

4、全文控制在 400 字以内，保持专业、简洁。

```
Issue: --name传入仅空白字符串时, 程序未报错退出
运行环境: Linux; Windows平台复现情况: 待确认
复现命令: `sdt-greet --name " "`
实际结果: 程序输出问候文本 `Hello, !`, 进程退出码为0, 没有触发错误终止。
期望结果: 当 `--name` 参数去除首尾空白后为空, 程序抛出 `SystemExit(2)` 退出, 不输出问候内容。

Git提交信息: 校验--name输入, 拦截全部由空白构成的姓名。
问题: 传入仅空格的name参数, 程序正常执行并输出无效问候, 退出码0。
解决方法: 解析参数后对name做空白去除校验; 若处理后为空, 则触发SystemExit(2)终止。

评审意见:
> 标签: Blocking
当前代码缺少对`--name`的空白输入校验。当用户传入全空格参数, 会生成无意义的问候输出, 同时返回成功退出码0。
风险: 上层调用脚本会误认为本次调用执行成功, 引发后续业务逻辑错误。
建议: 在参数解析完成后增加空白字符串校验; 补充单元测试, 覆盖仅空白输入场景, 确保触发SystemExit(2)。
```

图 10: communication.md

2.4 第四题：修复一个可复现的线性回归训练循环

1、补全训练循环、正确调用 zero_grad、backward 和 step。

```
luc@islowc-vm:~/q12$ cat > train.py <<'EOF'
> import torch
> from torch import nn
> torch.manual_seed(20260907)
> x = torch.linspace(-1, 1, 101).reshape(-1, 1)
> y = 3 * x - 1
> loss_fn = nn.MSELoss()
> model = nn.Linear(1, 1)
> opt = torch.optim.SGD(model.parameters(), lr=0.1)
> for _ in range(200):
>     pred = model(x)
>     loss = loss_fn(pred, y)
>     opt.zero_grad()
>     loss.backward()
>     opt.step()
```

图 11: train.py 训练循环部分

2、训练后调用 model.eval(), 并在 torch.no_grad() 中计算最终损失。

```
> model.eval()
> with torch.no_grad():
>     pred_final = model(x)
>     loss_final = loss_fn(pred_final, y)
>     weight = model.weight.item()
>     bias = model.bias.item()
>     print(f"final loss: {loss_final.item():.6f}")
>     print(f"weight: {weight:.4f}, bias: {bias:.4f}")
> EOF
```

图 12: model.eval() 与 torch.no_grad()

3、打印最终损失、weight 和 bias；最终损失应小于 0.001。

```
luc@islowc-vm:~/q12$ python3 train.py
final loss: 0.000000
weight: 3.0000, bias: -1.0000
```

图 13: 结果截图

由截图可以看出，损失确实小于 0.001

3 课外实例

3.1 实例一：用 printenv 对比虚拟环境激活前后的环境变量

- 1、使用 printenv 命令将当前的环境变量导出到文件中。

```
ouc@islouc-vm:~/exp1$ printenv > before.txt
ouc@islouc-vm:~/exp1$ echo "$PATH" > before_path.txt
```

图 14: 导出步骤

- 2、创建一个 Python 虚拟环境，并激活，再使用 printenv 将激活后的环境变量导出到另一个文件中。

```
ouc@islouc-vm:~/exp1$ python3 -m venv venv
ouc@islouc-vm:~/exp1$ source venv/bin/activate
(venv) ouc@islouc-vm:~/exp1$ which python3
/home/ouc/exp1/venv/bin/python3
(venv) ouc@islouc-vm:~/exp1$ echo "VIRTUAL_ENV=$VIRTUAL_ENV"
VIRTUAL_ENV=/home/ouc/exp1/venv
(venv) ouc@islouc-vm:~/exp1$ printenv > after.txt
(venv) ouc@islouc-vm:~/exp1$ echo "$PATH" > after_path.txt
```

图 15: 过程截图

- 3、使用 diff 命令对比 before.txt 和 after.txt 两个文件。

```
(venv) ouc@islouc-vm:~/exp1$ diff before.txt after.txt
30a31
> VIRTUAL_ENV=/home/ouc/exp1/venv
47a49
> (venv) \[\e]0;\u@h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]
\ue\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$
51c53
< PATH=/home/ouc/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
---
> PATH=/home/ouc/exp1/venv/bin:/home/ouc/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
55d56
< _=/usr/bin/printenv
56a58
> _=/usr/bin/printenv
```

图 16: diff 对比

对比 before.txt 与 after.txt, 变化有三处: 新增环境变量 VIRTUAL_ENV; 修改了提示符变量 PS1, 在原提示符前面拼接了 (venv) 前缀; 修改了 PATH: activate 把 /home/ouc/exp1/venv/bin 插入到了 PATH 的最前面。

- 4、为什么 shell 会优先使用这个 venv 虚拟环境?

shell 解析命令时, 会按 PATH 中目录从左到右的顺序依次查找第一个命中的可执行文件即为实际执行的程序, 激活虚拟环境后, /home/ouc/exp1/venv/bin 位于 PATH 的最左端, 所以会优先使用 venv 虚拟环境。

- 5、执行 which deactivate 命令, 推导分析 deactivate 这个 bash 内置函数实际在执行什么操作。

```
(venv) ouc@islouc-vm:~/exp1$ which deactivate
(venv) ouc@islouc-vm:~/exp1$ type deactivate
deactivate is a function
deactivate ()
{
    if [ -n "${_OLD_VIRTUAL_PATH:-}" ]; then
        PATH="${_OLD_VIRTUAL_PATH:-}";
        export PATH;
        unset _OLD_VIRTUAL_PATH;
    fi;
    if [ -n "${_OLD_VIRTUAL_PYTHONHOME:-}" ]; then
        PYTHONHOME="${_OLD_VIRTUAL_PYTHONHOME:-}";
        export PYTHONHOME;
        unset _OLD_VIRTUAL_PYTHONHOME;
    fi;
    if [ -n "${BASH:-}" -o -n "${ZSH_VERSION:-}" ]; then
        hash -r;
    fi;
    if [ -n "${_OLD_VIRTUAL_PS1:-}" ]; then
        PS1="${_OLD_VIRTUAL_PS1:-}";
        export PS1;
        unset _OLD_VIRTUAL_PS1;
    fi;
    unset VIRTUAL_ENV;
    if [ ! "${1:-}" = "nondestructive" ]; then
        unset -f deactivate;
    fi
}
```

图 17: which deactivate 命令

which deactivate 没有任何输出，说明 deactivate 不是一个独立的可执行文件；type deactivate 显示它是一个 bash 函数，是在执行 source venv/bin/activate 时被动态定义并注入当前 shell 的。deactivate 就是把 activate 当时对 shell 环境所做的所有修改逐项回滚，最后把自己也清理掉，使 shell 完全恢复到激活虚拟环境之前的状态。

3.2 实例二：写一个 Python 包，用 pyproject.toml 装进 venv，再生成锁文件

- 1、做一个有源码、有元数据的最小 Python 包：

```
(venv) ouc@islouc-vm:~/exp2$ mkdir exp2
(venv) ouc@islouc-vm:~/exp2$ cat > exp2/greet.py << 'EOF'
> def hello(name: str = "world") -> str:
>     return f"Hello, {name}! (from exp2)"
> EOF
(venv) ouc@islouc-vm:~/exp2$ cat > exp2/__init__.py << 'EOF'
> from .greet import hello
> __all__ = ["hello"]
> EOF
```

图 18: python 包的建立

- 2、将 pyproject.toml 写在外层

```
(venv) ouc@islouc-vm:~/exp2$ cat > pyproject.toml << 'EOF'
> [build-system]
> requires = ["setuptools>=61"]
> build-backend = "setuptools.build_meta"
> [project]
> name = "exp2"
> version = "0.1.0"
> description = "A tiny demo package for exp 3"
> requires-python = ">=3.8"
> dependencies = [
>     "requests>=2.31.0"
> ]
> EOF
(venv) ouc@islouc-vm:~/exp2$ ls -R
.:
exp2  pyproject.toml

./exp2:
greet.py  __init__.py
```

图 19: pyproject.toml

3、执行 `pip install -e .` 将本包以可编辑模式安装进 `venv`

```
(venv) ouc@islouc-vm:~/exp2$ pip install -e .
Obtaining file:///home/ouc/exp2
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting requests>=2.31.0 (from exp2==0.1.0)
  Downloading requests-2.32.4-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests>=2.31.0->exp2==0.1.0)
  Downloading charset_normalizer-3.5.1-cp37-abi3-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl.metadata (45 kB)
Collecting idna<4,>=2.5 (from requests>=2.31.0->exp2==0.1.0)
  Downloading idna-3.15-py3-none-any.whl.metadata (7.7 kB)
Collecting urllib3<3,>=1.21.1 (from requests>=2.31.0->exp2==0.1.0)
  Downloading urllib3-2.2.3-py3-none-any.whl.metadata (6.5 kB)
Collecting certifi>=2017.4.17 (from requests>=2.31.0->exp2==0.1.0)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.32.4-py3-none-any.whl (64 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Downloading charset_normalizer-3.5.1-cp37-abi3-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl (253 kB)
Downloading idna-3.15-py3-none-any.whl (72 kB)
Downloading urllib3-2.2.3-py3-none-any.whl (126 kB)
Building wheels for collected packages: exp2
  Building editable for exp2 (pyproject.toml) ... done
  Created wheel for exp2: filename=exp2-0.1.0-0.editable-py3-none-any.whl size=2622 sha256=adebc4d5c0d0355475198145651bac53166458b1f8e135b395322bfab981948b
    Stored in directory: /tmp/pip-ephem-wheel-cache-g0k79e8d/wheels/a1/61/45/c6330a584ffbb6f41bf91599d309190de21766c18decab0d29
Successfully built exp2
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests, exp2
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.1 exp2-0.1.0 idna-3.15 requests-2.32.4 urllib3-2.2.3
```

图 20: 结果截图

4、运行 `python -c "from exp2 import hello; print(hello('ouc'))"` 验证这个包真的能被 import。

```
(venv) ouc@islouc-vm:~/exp2$ python -c "from exp2 import hello; print(hello('ouc'))"
Hello, ouc! (from exp2)
```

图 21: 结果截图

5、生成一个锁文件并检查产物。

```
(venv) ouc@islouc-vm:~/exp2$ pip-compile pyproject.toml -o requirements-lock.txt
WARNING: --strip-extras is becoming the default in version 8.0.0. To silence this
warning, either use --strip-extras to opt into the new default or use --no-strip-e
xtras to retain the existing behavior.
#
# This file is autogenerated by pip-compile with Python 3.8
# by the following command:
#
#   pip-compile --output-file=requirements-lock.txt pyproject.toml
#
certifi==2026.7.22
#   via requests
charset-normalizer==3.5.1
#   via requests
idna==3.15
#   via requests
requests==2.32.4
#   via exp2 (pyproject.toml)
urllib3==2.2.3
#   via requests
(venv) ouc@islouc-vm:~/exp2$ cat requirements-lock.txt
#
# This file is autogenerated by pip-compile with Python 3.8
# by the following command:
#
#   pip-compile --output-file=requirements-lock.txt pyproject.toml
#
certifi==2026.7.22
#   via requests
charset-normalizer==3.5.1
#   via requests
idna==3.15
#   via requests
requests==2.32.4
#   via exp2 (pyproject.toml)
urllib3==2.2.3
#   via requests
```

图 22: 生成锁文件并检查产物

3.3 实例三：使用 Docker 与 Docker Compose 部署 Flask-Redis 多容器应用

1、创建一个 app.py，实现一个基于 Flask 的访问计数器。Redis 地址通过环境变量 REDIS_HOSE，默认为服务名 redis。

```
(venv) ouc@islouc-vm:~/exp3$ cat > app.py << 'EOF'
> import os
> import redis
> from flask import Flask
> app = Flask(__name__)
> r = redis.Redis(
>     host=os.getenv("REDIS_HOST", "redis"),
>     port=6379,
>     decode_responses=True,
> )
> @app.route("/")
> def index():
>     cnt = r.incr("hits")
>     return f"Hello from container! This page has been viewed {cnt} times.\n"
> if __name__ == "__main__":
>     app.run(host="0.0.0.0", port=5000)
> EOF
```

图 23: app.py

2、创建 requirements.txt，声明应用所需的第三方包：

```
(venv) ouc@islouc-vm:~/exp3$ cat > requirements.txt << 'EOF'
> flask
> redis
> EOF
```

图 24: requirements.txt

3、创建 Dockerfile，基于 python:3.11-alpine 构建应用镜像。为充分利用 Docker 构建缓存，先单独拷贝 requirements.txt 并安装依赖，再拷贝应用代码：

```
(venv) ouc@islouc-vm:~/exp3$ cat > Dockerfile << 'EOF'
> FROM python:3.11-slim
> WORKDIR /app
> COPY requirements.txt .
> RUN pip install --no-cache-dir -r requirements.txt
> COPY app.py .
> EXPOSE 5000
> CMD ["python", "app.py"]
> EOF
```

图 25: Dockerfile

4、编写 docker-compose.yml，定义两个服务：web 和 redis。

```
(venv) ouc@islouc-vm:~/exp3$ cat > docker-compose.yml << 'EOF'
> services:
>   web:
>     build: .
>     ports:
>       - "5000:5000"
>     environment:
>       - REDIS_HOST=redis
>     depends_on:
>       - redis
>   redis:
>     image: redis:7-alpine
> EOF
```

图 26: docker-compose.yml

5、配置镜像加速。

```
=> [web] exporting to image                                0.1s
=> => exporting layers                                      0.1s
=> => writing image sha256:be2af890420050cf57ab29a088aeb6f4d34a58a6b0cc2c4 0.0s
=> => naming to docker.io/library/exp3-web                 0.0s
[+] Running 3/3
✓ Network exp3_default      Created                                0.1s
✓ Container exp3-redis-1    Started                               0.0s
✓ Container exp3-web-1      Started                               0.0s
```

图 27: 实验截图

6、功能验证，连续三次访问 5000 端口，观察计数值递增。

```
ouc@islouc-vm:~/exp3$ curl http://localhost:5000/
Hello from container! This page has been viewed 1 times.
ouc@islouc-vm:~/exp3$ curl http://localhost:5000/
Hello from container! This page has been viewed 2 times.
ouc@islouc-vm:~/exp3$ curl http://localhost:5000/
Hello from container! This page has been viewed 3 times.
```

图 28: 结果截图

进入 Redis 容器，使用 redis-cli 直接读取 hits 键，验证数据确实存储在 Redis 中：

```
Hello from container! This page has been viewed 3 times.
ouc@islouc-vm:~/exp3$ docker compose exec redis redis-cli GET hits
"3"
```

图 29: 实验截图

3.4 实例四：注释查看

1、通过 Gitee 镜像克隆 Redis 源码

```
ouc@islouc-vm:~/exp3$ cd ~
ouc@islouc-vm:~$ git clone --depth 1 https://gitee.com/mirrors/redis.git
Cloning into 'redis'...
remote: Enumerating objects: 1792, done.
remote: Counting objects: 100% (1792/1792), done.
remote: Compressing objects: 100% (1371/1371), done.
remote: Total 1792 (delta 480), reused 1188 (delta 383), pack-reused 0 (from 0)
Receiving objects: 100% (1792/1792), 4.15 MiB | 2.14 MiB/s, done.
Resolving deltas: 100% (480/480), done.
```

图 30: 克隆截图

2、找出讲义中提到的几种注释类型

类型一：有用的 TODO

```
/* else if (r->type == REDIS_REPLY_PUSH) numsep = '>'; */
/* TODO: this would be a breaking change for scripts, do that in a maj
or version. */
/* else if (r->type == REDIS_REPLY_PUSH) numsep = '>'; */
else numsep = ')';
```

图 31: 注释原文

类型二：指向外部文档的引用

```
* Escape a Unicode string for JSON output (--json), following RFC 7159:
* https://datatracker.ietf.org/doc/html/rfc7159#section-7
*/
sds escapeJsonString(sds s, const char *p, size_t len) {
```

图 32: 注释原文

类型三：“Why not” 设计决策注释

```
* reoccur. One thought might be, why not to extend sds to support metadata. The
* answer is that sds is mutable string in its nature, with wide API (split, join,
* etc.). Pushing metadata logic into sds will make it very fragile, and complex
* to maintain.
```

图 33: 注释原文

类型四：血泪教训

```
/* Check if we need to do fsync even the aof buffer is empty,
* because previously in AOF_FSYNC_EVERYSEC mode, fsync is
* called only when aof buffer is not empty, so if users
* stop write commands before fsync called in one second,
* the data in page cache cannot be flushed in time. */
```

图 34: 注释原文

如果这些注释不存在，维护者将失去“为什么”的上下文：TODO 变成孤立的待办，设计决策看起来像随意选择，血泪教训看起来像冗余代码。注释是为了记录代码本身无法表达的信息。

3.5 实例五：三个高 star 项目 README 对比分析

- 1、从 Gitee 镜像克隆三个项目到本地，提取其 README.md：

```
(venv) ouc@islouc-vm:~/exp5$ ls -la *.md
-rw-rw-r-- 1 ouc ouc 74271 9月 16 00:57 awesome-python.md
-rw-rw-r-- 1 ouc ouc 35850 9月 16 00:47 redis.md
-rw-rw-r-- 1 ouc ouc 2929 9月 16 00:58 requests.md
```

图 35: 结果截图

2、它们都同样有用吗？并不是同样有用的。requests 的 README 最有效，首屏直接给可运行代码示例，读者能在最短时间内判断并上手；redis 的 README 信息全面但过于冗长，把本应独立的文档内容都堆了进来，首屏效率被稀释；awesome-python 作为资源列表，比较长。

3、在这三个文件中，最明显的噪音就是赞助和广告位，以及重复冗余信息和过多的徽章。

4、将来写 README 提供的借鉴：首屏用一句话说清项目是什么，紧接着给出可直接复制运行的代码示例，并附上指向完整文档的链接，而不是堆砌冗长的介绍或把编译指南、贡献流程、配置说明等本应独立成篇的内容全部塞进 README。同时应当避免在首屏放置赞助广告。

3.6 实例六：将 Python 包发布到 TestPyPI，并构建 Docker 镜像

- 1、将第 2 题开发的包改名为 ouc-exp6，使用 build 工具打包：

```
(venv) ouc@islouc-vm:~/exp2$ cd ~/exp2
(Burp Suite Community Edition) /M:~/exp2$ grep "^name" pyproject.toml
name = "exp2"
(venv) ouc@islouc-vm:~/exp2$ sed -i 's/^name = "exp2"/name = "ouc-exp6"/' pyproject.toml
(venv) ouc@islouc-vm:~/exp2$ grep "^name" pyproject.toml
name = "ouc-exp6"
(venv) ouc@islouc-vm:~/exp2$ pip install build twine
Requirement already satisfied: build in /home/ouc/exp1/venv/lib/python3.8/site-packages (1.2.2.post1)
Collecting twine
  Downloading twine-6.1.0-py3-none-any.whl.metadata (3.7 kB)
```

图 36: 实验过程

```
Cffi-1.17.1 cryptography-47.0.0 docutils-0.20.1 id-1.5.0 importlib-resources-6.4.5 jaraco.classes-3.4.0 jaraco.context-6.0.1 jaraco.functools-4.1.0 jeepney-0.9.0 keyring-25.5.0 markdown-it-py-3.0.0 mdurl-0.1.2 more-itertools-10.5.0 nh3-0.3.7 pycparser-2.23 readme-renderer-43.0 requests-toolbelt-1.0.0 rfc3986-2.0.0 rich-14.3.4 twine-6.1.0 typing-extensions-4.13.2
(venv) ouc@islouc-vm:~/exp2$ python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=61
* Getting build dependencies for sdist
```

图 37: 实验过程

- 2、生成两个产物

```
adding 'ouc_exp6-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built ouc_exp6-0.1.0.tar.gz and ouc_exp6-0.1.0-py3-none-any.whl
```

图 38: 产物

3、上传到 TestPyPI, 展示上传成功截图

```
(venv) ouc@islouc-vm:~/exp2$ python -m twine upload --repository testpypi dist/*
Uploading distributions to https://test.pypi.org/legacy/
/home/ouc/exp1/venv/lib/python3.8/site-packages/secretstorage/util.py:23: CryptographyDeprecationWarning: Python 3.8 is no longer supported by the Python core team
and support for it is deprecated in cryptography. The next release of cryptography
will remove support for Python 3.8.
  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
Enter your API token:
Uploading ouc_exp6-0.1.0-py3-none-any.whl
100% |#####| 3.1/3.1 kB • 00:00 • ?
Uploading ouc_exp6-0.1.0.tar.gz
100% |#####| 2.7/2.7 kB • 00:00 • ?

View at:
https://test.pypi.org/project/ouc-exp6/0.1.0/
```

图 39: 成功截图

4、从 TestPyPI 安装验证, 在新虚拟环境中从 TestPyPI 安装

```
(verifyenv) ouc@islouc-vm:~/tmp$ python -c "from exp2 import hello; print(hello('test'))"
Hello, test! (from exp2)
```

图 40: 实验截图

5、构建 Docker 镜像, 从 TestPyPI 安装该包

```
(verifyenv) ouc@islouc-vm:~/exp6$ docker build -t ouc-exp6 .
[+] Building 25.5s (6/6) FINISHED                                docker:default
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                   0.0s
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 367B                               0.0s
=> [internal] load metadata for docker.m.daocloud.io/library/python:3.11-a 0.0s
=> CACHED [1/2] FROM docker.m.daocloud.io/library/python:3.11-alpine 0.0s
=> [2/2] RUN pip install --no-cache-dir -i https://pypi.tuna.tsinghua.edu 25.4s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.1s
=> => writing image sha256:61002725a88880f9085017013faa0cd9d192e5355ea23a0 0.0s
=> => naming to docker.io/library/ouc-exp6                      0.0s
(verifyenv) ouc@islouc-vm:~/exp6$ docker run --rm ouc-exp6
Hello, docker! (from exp2)
```

图 41: 结果截图

其实原本需要将 ghcr.io, 但是由于我的虚拟机连不上 GitHub, 所以只能省略掉这一个步骤。

3.7 实例七: 用 docker compose 构建班级网站

1、在 /exp7 目录下创建了班级网站, 包含 website/index.html。这里 index.html 代码实在是太长了, 这里就不黏贴完整源代码了

2、Dockerfile 基于 nginx:alpine, 将静态文件拷贝到 /usr/share/nginx/html/。

```
(venv) ouc@islouc-vm:~/exp7$ cat > Dockerfile << 'EOF'
> FROM docker.m.daocloud.io/library/nginx:alpine
> COPY website/ /usr/share/nginx/html/
> EXPOSE 80
> EOF
```

图 42: Dockerfile

3、docker-compose.yml 定义 web 服务，构建当前目录的镜像并将容器 80 端口映射到宿主机 8080。

```
(venv) ouc@tslouc-vm:~/exp7$ cat > docker-compose.yml << 'EOF'
> services:
>   web:
>     build: .
>     ports:
>       - "8080:80"
> EOF
```

图 43: docker-compose.yml

4、通过 docker compose up -build -d 启动后，访问 http://127.0.0.1:8080 即可看到班级主页。

```
(venv) ouc@tslouc-vm:~/exp7$ curl http://127.0.0.1:8080/
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>一班班级主页</title>
  <style>
    body { font-family: "Microsoft YaHei", sans-serif; max-width: 800px; margin: 40px auto; padding: 0 20px; background: #f5f5f5; }
    header { background: #2c3e50; color: white; padding: 30px; border-radius: 8px; text-align: center; }
    section { background: white; padding: 20px; margin: 20px 0; border-radius: 8px; box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1); }
    h2 { color: #2c3e50; border-bottom: 2px solid #3498db; padding-bottom: 8px; }
    table { width: 100%; border-collapse: collapse; }
    th, td { padding: 10px; border: 1px solid #ddd; text-align: center; }
    th { background: #3498db; color: white; }
    ul { line-height: 1.8; }
  </style>
</head>
<body>
  <header>
    <h1>计算机科学与技术 一班</h1>
    <p>欢迎来到我们的班级主页</p>
  </header>
  <section>
    <h2>班级简介</h2>
    <p>一班共有学生35人，其中男生28人，女生7人。本学期开设课程包括操作系统、计算机网络、编译原理、软件工程等。</p>
  </section>
  <section>
    <h2>本周课程表</h2>
    <table>
      <tr><th>时间</th><th>周一</th><th>周二</th><th>周三</th><th>周四</th><th>周五</th></tr>
      <tr><td>1-2节</td><td>操作系统</td><td>计算机网络</td><td>编译原理</td><td>软件工程</td><td>操作系统</td></tr>
      <tr><td>3-4节</td><td>计算机网络</td><td>编译原理</td><td>操作系统</td><td>计算机网络</td><td>体育</td></tr>
      <tr><td>5-6节</td><td>编译原理</td><td>操作系统</td><td>软件工程</td><td>自习</td><td>编译原理</td></tr>
    </table>
  </section>
</body>
```

图 44: 结果截图

3.8 实例八：用 GitHub Pages 创建网站

1、由于虚拟机连不上 GitHub,所以我们依旧使用 Gitee, 新建公开仓库 class-website, 将第 7 题编写的班级主页 index.html 推送至仓库 master 分支：

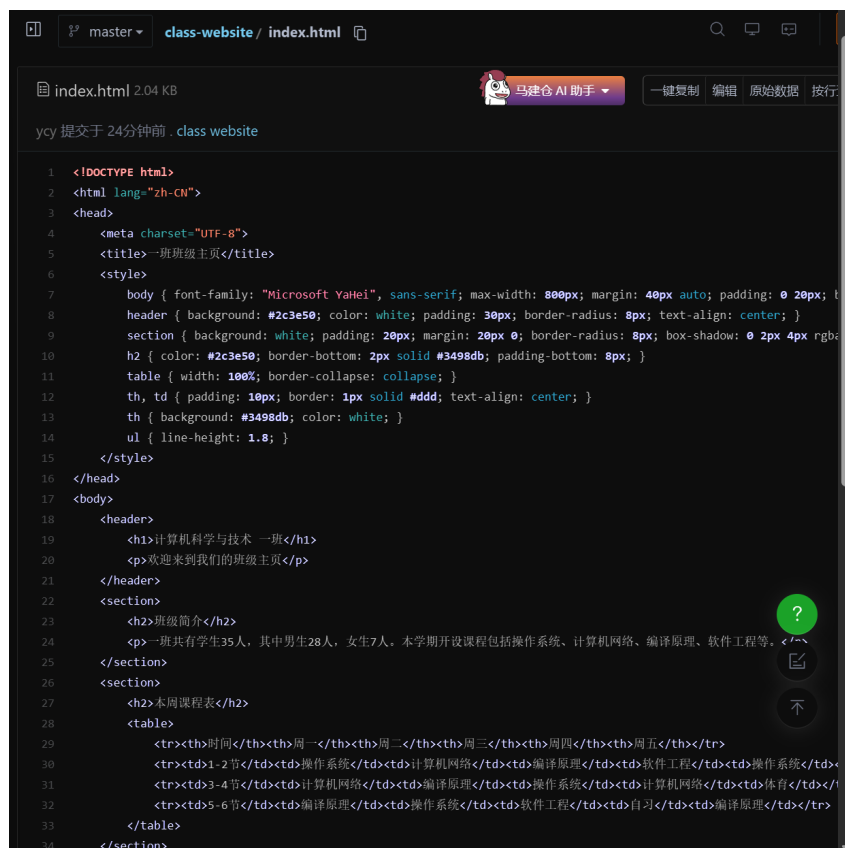


图 45: 页面截图

3.9 实例九：为编程智能体创建 AGENTS.md、Skill 与子智能体

1、AGENTS.md，项目说明文件，放在项目根目录，AI 助手每次开始工作时会自动读取。内容是这个项目的硬性约定：用什么测试框架、怎么跑测试、代码风格、目录结构、提交规则。

```
(venv) ouc@islouc-vm:~/exp9/my-project$ cat > AGENTS.md << 'EOF'
> # 项目说明 (给 AI 编程助手看)
> ## 项目概览
> 这是一个 Python 学习项目，用于练习模块开发和测试。
> ## 构建与测试
> - 运行测试：`python3 -m pytest tests/ -v`
> - 代码风格：遵循 PEP 8，行宽 88 字符
> - 不使用 Python 2 语法
> Software Updater 同
> - `src/`：业务代码
> - `tests/`：pytest 测试
> - `docs/`：文档
> ## 规则
> 1. 写新功能前先写测试
> 2. 函数必须有类型标注
> 3. 不要在主分支直接提交，先开分支
> EOF
```

图 46: AGENTS.md

作用：让 AI 知道”在这个项目里应该怎么做”，而不是每次都重新问。

2、Skill，描述一个可复用的动作模板，比如 code-review 技能定义了”审查代码时检查什么、什么时候触发、怎么输出”。

```
(venv) ouc@islouc-vm:~/exp9/my-project$ cat > .agents/skills/code-review/SKILL.md
<< 'EOF'
> # Skill: code-review
> ## 描述
> 审查一段 Python 代码，检查：
> - 是否有类型标注
> - 是否有对应的测试
> - 是否遵循 PEP 8
> - 函数是否过长（超过 30 行提示拆分）
> ## 使用时机
> 当用户要求"审查代码"、"review"、"看看这段代码有没有问题"时触发。
> ## 输出格式
> 1. 列出发现的问题（按严重程度排序）
> 2. 给出修改建议
> 3. 没有问题时明确说"通过"
> EOF
```

图 47: Skills

作用：把一套反复使用的操作流程固化下来，不用每次都重新描述要求。

3、子智能体，定义一个有独立角色和职责的专门智能体。比如 test-writer 子智能体只负责写测试，不修改源代码。

```
(venv) ouc@islouc-vm:~/exp9/my-project$ cat > .agents/agents/test-writer/AGENT.md
<< 'EOF'
> # 子智能体：test-writer
> ## 角色
> 你是一个专门写测试的智能体。
> ## 职责
> - 为给定的 Python 函数编写 pytest 测试
> - 覆盖正常情况、边界情况、异常情况
> - 不修改源代码，只写测试文件
> ## 输入
> 用户提供源代码文件路径。
> ## 输出
> 在 tests/ 目录下创建对应的测试文件，并简要说明覆盖了哪些场景。
> EOF
(venv) ouc@islouc-vm:~/exp9/my-project$ cat > .agents/main.md << 'EOF'
> # 主智能体配置
> ## 子智能体
> - test-writer：当用户要求"写测试"、"补测试"、"增加覆盖率"时，调用 test-writer 子智能体
> - code-review：当用户要求"审查代码"、"review"时，使用 code-review Skill
> ## 默认行为
> - 回答问题前先读 AGENTS.md
> - 不确定项目约定时，问用户而不是猜
> EOF
```

图 48: 子智能体

作用：把复杂任务拆分，让专门的智能体在自己的上下文里工作，避免主对话被细节淹没。

3.10 实例十：氛围编程

- 1、和 AI 对话，让 AI 编写一个命令行待办事项管理器

```
(venv) ouc@islouc-vm:~/exp10$ cat > todo.py << 'EOF'
Google Chrome
> import sys
> from pathlib import Path
>
> TODO_FILE = Path.home() / ".todo_list.json"
>
> def load_todos():
>     if TODO_FILE.exists():
>         with open(TODO_FILE, "r", encoding="utf-8") as f:
>             return json.load(f)
>         return []
>
> def save_todos(todos):
>     with open(TODO_FILE, "w", encoding="utf-8") as f:
>         json.dump(todos, f, ensure_ascii=False, indent=2)
>
> def add_task(text):
>     todos = load_todos()
>     todos.append({"task": text, "done": False})
>     save_todos(todos)
>     print(f"已添加: {text}")
>
> def list_tasks():
>     todos = load_todos()
>     if not todos:
>         print("暂无任务")
>         return
>     for i, t in enumerate(todos, 1):
>         mark = "[x]" if t["done"] else "[ ]"
>         print(f"{i}. {mark} {t['task']}")
>
> def done_task(index):
>     todos = load_todos()
>     if 1 <= index <= len(todos):
>         todos[index - 1]["done"] = True
>         save_todos(todos)
>         print(f"已完成: {todos[index - 1]['task']}")
>     else:
>         print("序号不存在")
>
> def delete_task(index):
>     todos = load_todos()
>     if 1 <= index <= len(todos):
>         removed = todos.pop(index - 1)
>         save_todos(todos)
>         print(f"已删除: {removed['task']}")
>     else:
>         print("序号不存在")
>
> def main():
>     if len(sys.argv) < 2:
>         print("用法: python todo.py <add|list|done|delete> [内容/序号]")
>         return
>     cmd = sys.argv[1]
>     if cmd == "add":
>         add_task(" ".join(sys.argv[2:]))
>     elif cmd == "list":
>         list_tasks()
>     elif cmd == "done":
>         done_task(int(sys.argv[2]))
>     elif cmd == "delete":
>         delete_task(int(sys.argv[2]))
>     else:
>         print(f"未知命令: {cmd}")
>
> if __name__ == "__main__":
>     main()
> EOF
```

图 49: 完整代码

2、运行验证

```
(venv) ouc@islouc-vm:~/exp10$ cd ~/exp10
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py add 买牛奶
已添加: 买牛奶
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py add 写实验报告
已添加: 写实验报告
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py add 跑步30分钟
已添加: 跑步30分钟
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py list
1. [ ] 买牛奶
2. [ ] 写实验报告
3. [ ] 跑步30分钟
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py done 2
已完成: 写实验报告
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py list
1. [ ] 买牛奶
2. [x] 写实验报告
3. [ ] 跑步30分钟
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py delete 1
已删除: 买牛奶
(venv) ouc@islouc-vm:~/exp10$ python3 todo.py list
1. [x] 写实验报告
2. [ ] 跑步30分钟
```

图 50: 结果截图

4 解题感悟

通过这次实验，我学会了用 `venv` 隔离环境、用 `pyproject.toml` 打包并生成锁文件、用 `Docker` 和 `docker-compose` 编排多容器应用，还阅读了 `Redis` 源码中的高质量注释，对比了知名项目的 `README` 写法。过程中遇到镜像拉取慢、虚拟环境机制不理解、容器间网络通信等问题，通过查文档和调试逐一解决，深刻体会到了工具链在实际开发中的价值，也认识到好的注释和文档对项目维护的重要性。